

Compressed Unet

July 13, 2019

```
In [1]: from models import *
```

```
In [2]: from unet import *
```

0.1 Methods

```
In [3]: def fit(model,opt,epoch,dl,crit):

    for j in trange(epoch,desc="Epoch",position=1,mininterval=0):
        train_loss = 0
        val_loss = 0

        one_zero_trn = 0.0
        one_zero_val = 0.0

        k = 0
        t=tqdm(iter(dl.trn_dl), leave=False, total=len(dl.trn_dl), miniters=0 ,postfix
        model.train()
        for data in t:
            k+=1

            opt.zero_grad()

            data = data.type(torch.LongTensor).cuda()
            out , img = model(V(data))

            img = img[:,0,:,:]

            #temp_data = data[:,0,:,:].reshape(data.shape[0],64,64).type(torch.).cuda()
            one_zero_trn = out.sum() / img.sum()

            loss = crit(out,img)
            loss.backward()
            opt.step()
            train_loss += loss.data
```

```

t.close()
t.set_postfix_str(f'loss : {float(train_loss)/k} 1:1 :{one_zero_trn}',refl

k = 0
with torch.no_grad():
    model.eval()
    t=tqdm(iter(dl.val_dl), leave=False, total=len(dl.val_dl), miniters=0,post:
    for data in t:
        k+=1

        data = data.type(torch.LongTensor).cuda()
        out , img = model(V(data))
        img = img[:,0,:,:]

        #print(data[:,0,:,:].reshape(16,1,256,256).shape)
        #temp_data = data[:,0,:,:].reshape(data.shape[0],64,64).type(torch.Lon
        one_zero_val = out.sum() / img.sum()
        model.one_to_one(one_zero_val)

        loss = crit(out,img)
        val_loss += loss

    t.close()
    t.set_postfix_str(f'loss : {float(val_loss)/k} 1:1 :{one_zero_val}',.

    model.record(loss)
    train_loss /= len(dl.trn_dl)
    val_loss /= len(dl.val_dl)
    print('train Loss: ', train_loss.tolist(), ' validation loss: ',val_loss.to

```

0.1.1 Customized Dataset

```

In [40]: class dataset(Dataset):
    def __init__(self,base_dir , index ,dim, mode):
        super().__init__()

        files = os.listdir(base_dir)
        files = files

        names = [files[i] for i in index]
        self.data = [ base_dir +i for i in names]

        self.mode = mode

        self.dim = dim
        self.total_length = dim*dim

```

```

def __getitem__(self, index):

    example = np.load(self.data[index]).astype(np.float)

    dim_example = len(example)
    diff = self.total_length - dim_example
    if self.mode == 1:

        left_pad = diff // 2
        right_pad = diff - left_pad
    else:
        left_pad = 0
        right_pad = diff
    example = np.pad(example, (left_pad, right_pad), 'constant').reshape(1, self.dim, self.dim)

    example = np.concatenate([example, example, example], axis = 0)

    return example

def __len__(self):
    return len(self.data)
    raise NotImplementedError

```

0.1.2 Split Data Train/Val

```

In [5]: def split(base_dir, dim, prc=0.2 , dataset_pct = 1, mode = 0):

    total_size = int(len(os.listdir(base_dir)) * dataset_pct)

    val_size = int(total_size*prc)

    trn_index = np.arange(total_size - val_size)
    val_index = np.arange(trn_index[-1]+1, total_size)

    val = dataset(base_dir , val_index, dim , mode = mode)
    trn = dataset(base_dir , trn_index, dim, mode = mode)
    return trn , val

In [6]: trn , val = split('../..//dataset/hex/', dim=64 , dataset_pct=0.1, mode= 0)

```

0.1.3 Prepare DataLoaders

```

In [7]: trn_dl = DataLoader(trn, batch_size=8, num_workers=2, shuffle=False)
        val_dl = DataLoader(val, batch_size=8, num_workers=2, shuffle=False)

In [8]: dl = CombineDataLoader(trn_dl, val_dl)

```

0.2 Model

```
In [9]: class U(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.one = []
            ##Load pre-trained
            self.deconv1 = nn.ConvTranspose2d(3,3,(65,65))
            self.deconv2 = nn.ConvTranspose2d(3,3,(65,65))
            self.deconv3 = nn.ConvTranspose2d(3,3,(65,65))

            self.upSampler = nn.ModuleList([self.deconv1 , self.deconv2,self.deconv3])

            self.conv1 = nn.Conv2d(1,16,(65,65))
            self.conv2 = nn.Conv2d(16,16,(65,65))
            self.conv3 = nn.Conv2d(16,16,(65,65))

            self.downSampler = nn.ModuleList([self.conv1 , self.conv2,self.conv3])

            self.unet = UNet().cuda()
            state_dict = torch.load('weights/unet.pt', map_location="cuda:0")
            self.unet.load_state_dict(state_dict)

            self.baseModel = nn.ModuleList([self.unet])

        def forward(self,inputs):
            k = f.elu(self.deconv1(inputs))
            k = f.elu(self.deconv2(k))
            k = f.elu(self.deconv3(k))

            k = self.unet(k)

            k = f.elu(self.conv1(k))
            k = f.elu(self.conv2(k))
            k = f.softmax(self.conv3(k),dim=1)
            return k

        def record(self,record):
            self.loss.append(record)

        def one_to_one(self,record):
            self.one.append(record)
```

```
In [10]: m = U().cuda()
```

```
In [11]: opt = torch.optim.Adam(m.parameters(),1e-3 /2)
```

```

In [12]: for i in m.baseModel.parameters():
          i.requires_grad_(False)

In [13]: fit(m,opt,1,dl,f.cross_entropy)

HBox(children=(IntProgress(value=0, description='Epoch', max=1, style=ProgressStyle(description

100%|| 267/267 [02:51<00:00, 1.99it/s, loss : 1.9086779190806413 1:1 :5.0288519859313965]

train Loss: 1.9104384183883667 validation loss: 1.9086779356002808


In [23]: torch.save(m,'model.pkl')

/home/amr/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/serialization.py:251: UserWarning:
  "type " + obj.__name__ + ". It won't be checked "

In [25]: a = torch.load('model.pkl')

In [27]: for i in m.baseModel.parameters():
          i.requires_grad_(True)

In [28]: for i in opt.param_groups:
          i['lr'] = 1e-4

In [29]: fit(m,opt,2,dl,f.cross_entropy)

HBox(children=(IntProgress(value=0, description='Epoch', max=2, style=ProgressStyle(description

100%|| 267/267 [02:51<00:00, 2.01it/s, loss : 1.9086779190806413 1:1 :5.0288519859313965]])

train Loss: 1.9094492197036743 validation loss: 1.9086779356002808


100%|| 267/267 [02:51<00:00, 2.00it/s, loss : 1.9086779190806413 1:1 :5.0288519859313965]])

train Loss: 1.9094492197036743 validation loss: 1.9086779356002808

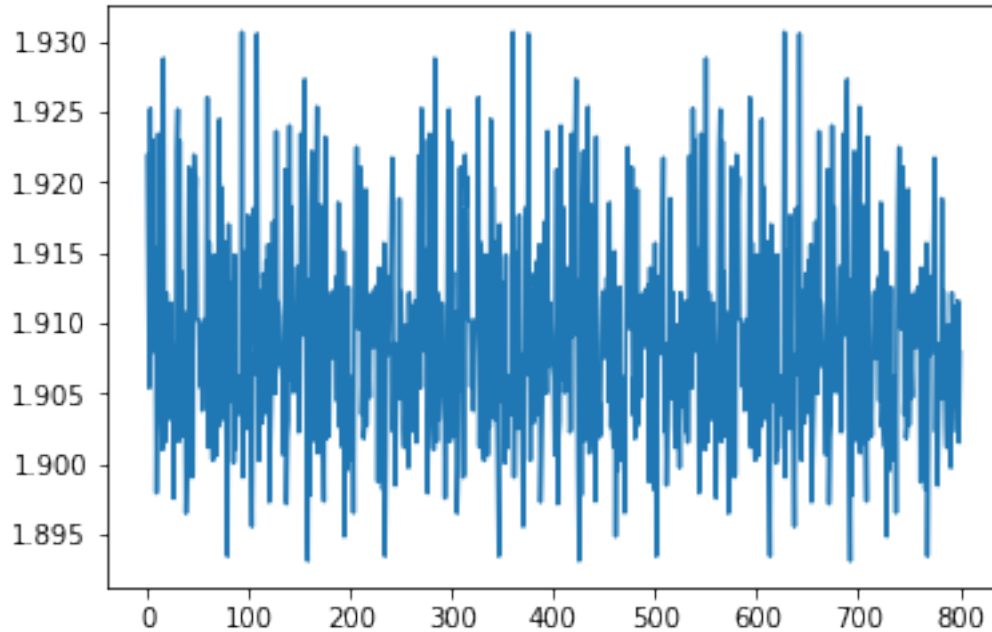

In [31]: torch.save(m , 'model_1.pkl')

/home/amr/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/serialization.py:251: UserWarning:
  "type " + obj.__name__ + ". It won't be checked "

In [32]: plt.plot(m.loss)

Out[32]: [matplotlib.lines.Line2D at 0x7f8c41ee9860>]

```



```
In [ ]:[:,0,:,:].reshape(16,1,64,64)
```

```
In [44]: k = next(iter(val_dl)).type(torch.float).cuda().permute(0,1,3,2)
```

```
In [45]: b = m(k)
```

```
In [46]: f.cross_entropy(b,k[:,0,:,:].type(torch.LongTensor).cuda())
```

```
Out[46]: tensor(1.9219, device='cuda:0', grad_fn=<NllLoss2DBackward>)
```

0.3 UNET with Embeddings

0.3.1 Customized Dataset

```
In [4]: class dataset(Dataset):
        def __init__(self,base_dir , index ,dim, mode):
            super().__init__()

            files = os.listdir(base_dir)
            files = files

            names = [files[i] for i in index]
            self.data = [ base_dir +i for i in names]

            self.mode = mode

            self.dim = dim
```

```

        self.total_length = dim*dim
    def __getitem__(self, index):

        example = np.load(self.data[index]).astype(np.float)

        dim_example = len(example)
        diff = self.total_length - dim_example
        if self.mode == 1:

            left_pad = diff // 2
            right_pad = diff - left_pad
        else:
            left_pad = 0
            right_pad = diff
        example = np.pad(example, (left_pad, right_pad), 'constant').reshape(1, self.dim*s

        example = np.concatenate([example, example, example], axis = 0)

        return example

    def __len__(self):
        return len(self.data)
        raise NotImplementedError

```

0.3.2 Split Data Train/Val

```

In [5]: def split(base_dir, dim, prc=0.2 , dataset_pct = 1, mode = 0):

        total_size = int(len(os.listdir(base_dir)) * dataset_pct)

        val_size = int(total_size*prc)

        trn_index = np.arange(total_size - val_size)
        val_index = np.arange(trn_index[-1]+1, total_size)

        val = dataset(base_dir , val_index, dim , mode = mode)
        trn = dataset(base_dir , trn_index, dim, mode = mode)
        return trn , val

In [6]: trn , val = split('.././dataset/hex/', dim=64 , dataset_pct=0.1, mode= 0)

```

0.3.3 Prepare DataLoaders

```

In [7]: trn_dl = DataLoader(trn, batch_size=8, num_workers=2, shuffle=False)
        val_dl = DataLoader(val, batch_size=8, num_workers=2, shuffle=False)

In [8]: dl = CombineDataLoader(trn_dl, val_dl)

```

```

In [9]: class Emb(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.one = []

            self.embedding = nn.Embedding(16,16)
            self.embedding.data = torch.eye(16,16)
        def forward(self,inputs):
            k = torch.sigmoid(self.embedding(inputs))

            return k.reshape(k.shape[0],k.shape[1],256,256)

        def record(self,record):
            self.loss.append(record)

        def one_to_one(self,record):
            self.one.append(record)

In [10]: class U(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.one = []
            ##Load pre-trained
            self.emb = Emb()

            self.embMod = nn.ModuleList([self.emb])

            self.unet = UNet().cuda()
            state_dict = torch.load('weights/unet.pt', map_location="cuda:0")
            self.unet.load_state_dict(state_dict)

            self.baseModel = nn.ModuleList([self.unet])

        def forward(self,inputs):

            k1 = self.emb(inputs).type(torch.float)

            k = self.unet(k1)

            return k , k1

        def record(self,record):
            self.loss.append(record)

        def one_to_one(self,record):

```



```

        self.one.append(record)

In [11]: m = U().cuda()

In [12]: opt = torch.optim.Adam(m.parameters(),1e-4)

In [13]: for i in m.baseModel.parameters():
            i.requires_grad_(False)

In [14]: fit(m,opt,1,dl,f.binary_cross_entropy)

HBox(children=(IntProgress(value=0, description='Epoch', max=1, style=ProgressStyle(description
0%|          | 0/1066 [00:00<?, ?it/s, Loss: 0]/home/amr/anaconda3/envs/fastv1/lib/python3.7/
"Please ensure they have the same size.".format(target.size(), input.size()))

```

```

-----

RuntimeError                                Traceback (most recent call last)

<ipython-input-14-dac23a64ed20> in <module>
----> 1 fit(m,opt,1,dl,f.binary_cross_entropy)

<ipython-input-3-3175434a0986> in fit(model, opt, epoch, dl, crit)
    25         one_zero_trn = out.sum() / img.sum()
    26
---> 27         loss = crit(out,img)
    28         loss.backward()
    29         opt.step()

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/functional.py in binary_c
2025
2026     return torch._C._nn.binary_cross_entropy(
-> 2027         input, target, weight, reduction_enum)
2028
2029

```

```

RuntimeError: the derivative for 'target' is not implemented

```